

PROGRAM

```
#include <stdio.h>

#include <conio.h>

#include <graphics.h>

#include <time.h>

void main()

{

    int gd = DETECT, gm;

    int arr[1000], i, n, idx = 0;

    int x[10], yBest[10], yAvg[10], yWorst[10];

    int key;

    clock_t start, end;

    double best, avg, worst;

    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");

    setcolor(WHITE);

    setlinestyle(SOLID_LINE, 0, 3);

    line(50, 400, 50, 50);

    line(50, 400, 600, 400);

    outtextxy(60, 40, "Time / Comparisons");

    outtextxy(480, 420, "Input Size (n)");

    printf("n\tBest\tAverage\tWorst\n");

    printf("-----\n");

    for (n = 10; n <= 100; n += 10)

    {

        for (i = 0; i < n; i++)

            arr[i] = i + 1;

        key = arr[0];

        start = clock();

        for (i = 0; i < n; i++)
```

```

        if (arr[i] == key) break;
    end = clock();
    best = (double)(end - start) / CLOCKS_PER_SEC;
    key = arr[n / 2];
    start = clock();
    for (i = 0; i < n; i++)
        if (arr[i] == key) break;
    end = clock();
    avg = (double)(end - start) / CLOCKS_PER_SEC;
    key = arr[n - 1];
    start = clock();
    for (i = 0; i < n; i++)
        if (arr[i] == key) break;
    end = clock();
    worst = (double)(end - start) / CLOCKS_PER_SEC;
    printf("%d\t%f\t%f\t%f\n", n, best, avg, worst);
    x[idx] = 50 + n * 5;
    yBest[idx] = 380;
    yAvg[idx] = 400 - (n * 2);
    yWorst[idx] = 400 - (n * 3);
    idx++;
}

setcolor(GREEN);
for (i = 0; i < idx - 1; i++)
    line(x[i], yBest[i], x[i+1], yBest[i+1]);
setcolor(BLUE);
for (i = 0; i < idx - 1; i++)
    line(x[i], yAvg[i], x[i+1], yAvg[i+1]);
setcolor(RED);

```

```

for (i = 0; i < idx - 1; i++)

    line(x[i], yWorst[i], x[i+1], yWorst[i+1]);

setcolor(GREEN); outtextxy(420, 80, "Best Case");

setcolor(BLUE); outtextxy(420, 100, "Average Case");

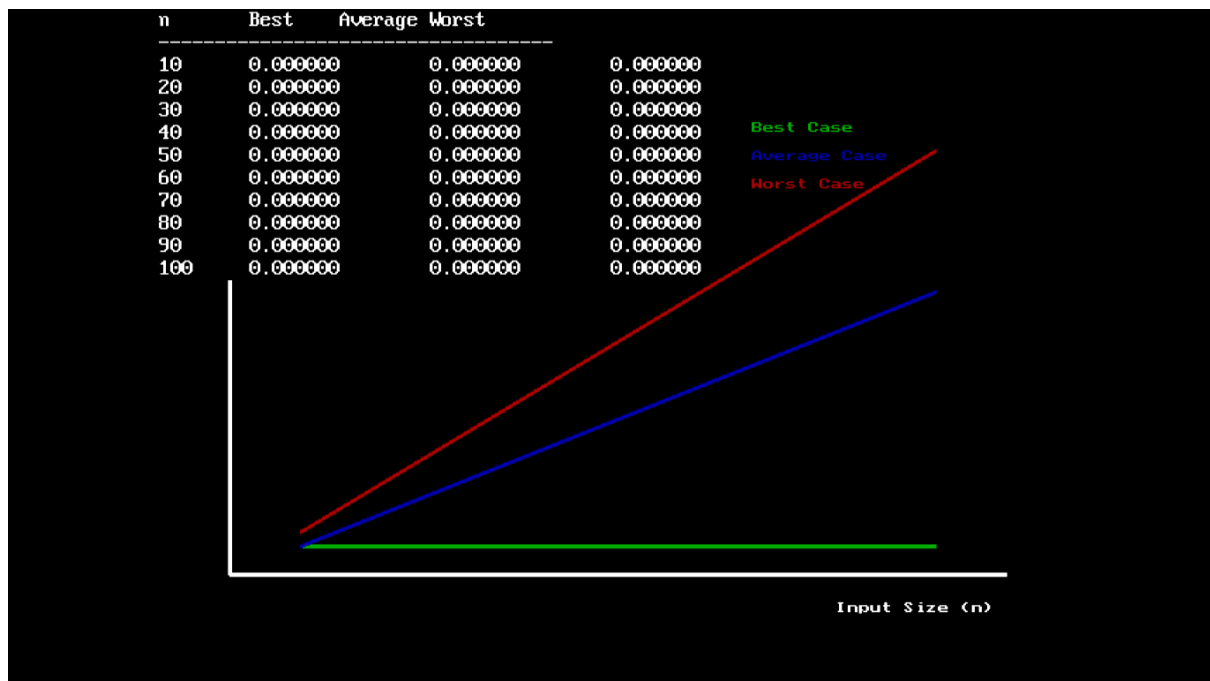
setcolor(RED); outtextxy(420, 120, "Worst Case");

getch();

closegraph();
}

```

OUTPUT



PROGRAM

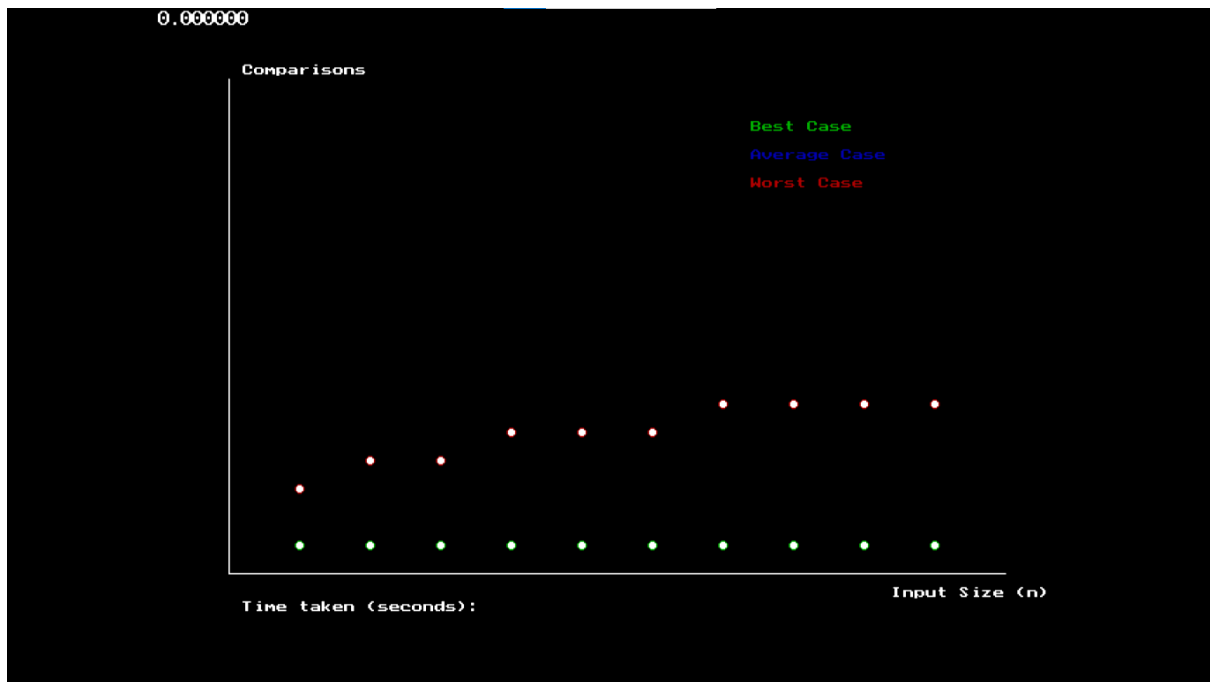
```
#include <graphics.h>
#include <conio.h>
#include <stdio.h>
#include <math.h>
#include <time.h>

int main()
{
    int gd = DETECT, gm;
    int n, x, y;
    int best, avg, worst;
    clock_t start, end;
    double time_taken;
    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");
    line(50, 400, 50, 50);
    line(50, 400, 600, 400);
    outtextxy(60, 40, "Comparisons");
    outtextxy(520, 410, "Input Size (n)");
    start = clock();
    for (n = 10; n <= 100; n += 10)
    {
        x = 50 + (n * 5);
        best = 1;
        avg = (int)(log(n) / log(2));
        worst = avg;
        setcolor(GREEN);
        circle(x, 400 - best * 20, 3);
        floodfill(x, 400 - best * 20, GREEN);
        setcolor(BLUE);
```

```
    circle(x, 400 - avg * 20, 3);
    floodfill(x, 400 - avg * 20, BLUE);
    setcolor(RED);
    circle(x, 400 - worst * 20, 3);
    floodfill(x, 400 - worst * 20, RED);
}

end = clock();
time_taken = (double)(end - start) / CLOCKS_PER_SEC;
setcolor(WHITE);
outtextxy(60, 420, "Time taken (seconds): ");
printf("%f", time_taken);
setcolor(GREEN);
outtextxy(420, 80, "Best Case");
setcolor(BLUE);
outtextxy(420, 100, "Average Case");
setcolor(RED);
outtextxy(420, 120, "Worst Case");
getch();
closegraph();
return 0;
}
```

OUTPUT



PROGRAM

```
#include <stdio.h>

#include <conio.h>

#include <string.h>

#define MAX 100

void computeLPSArray(char pat[], int m, int lps[])
{
    int len=0,i=1;
    lps[0]=0;
    while(i<m)
    {
        if(pat[i]==pat[len])
        {
            len++;
            lps[i]=len;
            i++;
        }
        else
        {
            if(len!=0)
                len=lps[len-1];
            else
            {
                lps[i]=0;
                i++;
            }
        }
    }
}
```

```

void search(char pat[], char txt[])
{
    int m=strlen(pat),n=strlen(txt);
    int lps[MAX];
    int i=0,j=0;
    computeLPSArray(pat,m,lps);
    while(i<n)
    {
        if(pat[j]==txt[i])
        {
            i++;
            j++;
        }
        if(j==m)
        {
            printf("Pattern found at index %d\n",i-j);
            j=lps[j-1];
        }
        else if(i<n && pat[j]!=txt[i])
        {
            if(j!=0)
                j=lps[j-1];
            else
                i++;
        }
    }
}

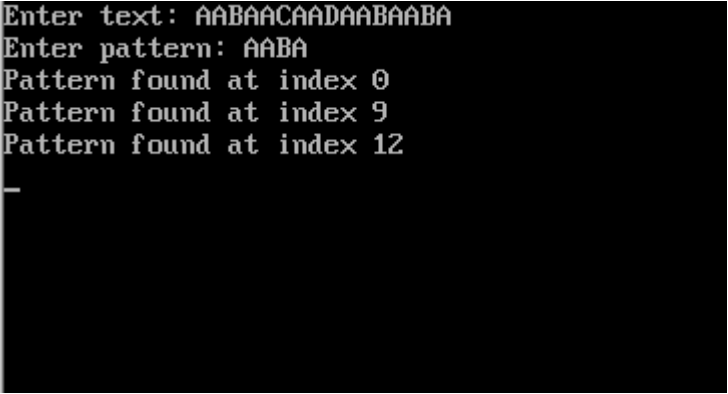
void main()
{

```



```
char txt[MAX],pat[MAX];  
  
clrscr();  
  
printf("Enter text: ");  
  
gets(txt);  
  
printf("Enter pattern: ");  
  
gets(pat);  
  
search(pat,txt);  
  
getch();  
  
}
```

OUTPUT

A screenshot of a terminal window with a black background and white text. The text shows the execution of a C program. It starts with 'Enter text: AABAACAADAABAABA', followed by 'Enter pattern: AABA'. Then, it displays three lines of output: 'Pattern found at index 0', 'Pattern found at index 9', and 'Pattern found at index 12'. A single underscore character is visible on the line following the last output.

```
Enter text: AABAACAADAABAABA  
Enter pattern: AABA  
Pattern found at index 0  
Pattern found at index 9  
Pattern found at index 12  
_
```

PROGRAM

```
#include <stdio.h>

#include <conio.h>

#include <graphics.h>

#include <time.h>

#include <stdlib.h>

void insertionSort(int arr[],int n)
{
    int i,key,j;
    for(i=1;i<n;i++)
    {
        key=arr[i];
        j=i-1;
        while(j>=0 && arr[j]>key)
        {
            arr[j+1]=arr[j];
            j--;
        }
        arr[j+1]=key;
    }
}

void heapify(int arr[],int n,int i)
{
    int largest=i,left=2*i+1,right=2*i+2,temp;
    if(left<n && arr[left]>arr[largest])
        largest=left;
    if(right<n && arr[right]>arr[largest])
        largest=right;
    if(largest!=i)
```

```

    {
        temp=arr[i];
        arr[i]=arr[largest];
        arr[largest]=temp;
        heapify(arr,n,largest);
    }
}

void heapSort(int arr[],int n)
{
    int i,temp;
    for(i=n/2-1;i>=0;i--)
        heapify(arr,n,i);
    for(i=n-1;i>0;i--)
    {
        temp=arr[0];
        arr[0]=arr[i];
        arr[i]=temp;
        heapify(arr,i,0);
    }
}

void main()
{
    int gd=DETECT,gm;
    int arr1[10000],arr2[10000];
    int n,i,idx=0;
    int x[10],yIns[10],yHeap[10];
    double ins_time[10],heap_time[10];
    clock_t start,end;
    srand(time(NULL));

```

```
initgraph(&gd,&gm,"C:\\TURBOC3\\BGI");
setcolor(WHITE);
setlinestyle(SOLID_LINE,0,3);
line(50,400,50,50);
line(50,400,600,400);
outtextxy(60,40,"Time (seconds)");
outtextxy(450,420,"Input Size (n)");
printf("n\\tInsertion\\tHeap\\n");
printf("-----\\n");
for(n=1000;n<=5000;n+=1000)
{
    for(i=0;i<n;i++)
    {
        arr1[i]=rand();
        arr2[i]=arr1[i];
    }
    start=clock();
    insertionSort(arr1,n);
    end=clock();
    ins_time[idx]=(double)(end-start)/CLOCKS_PER_SEC;
    start=clock();
    heapSort(arr2,n);
    end=clock();
    heap_time[idx]=(double)(end-start)/CLOCKS_PER_SEC;
    printf("%d\\t%f\\t%f\\n",n,ins_time[idx],heap_time[idx]);
    x[idx]=50+(n/10);
    yIns[idx]=400-(ins_time[idx]*200);
    yHeap[idx]=400-(heap_time[idx]*200);
    idx++;
}
```

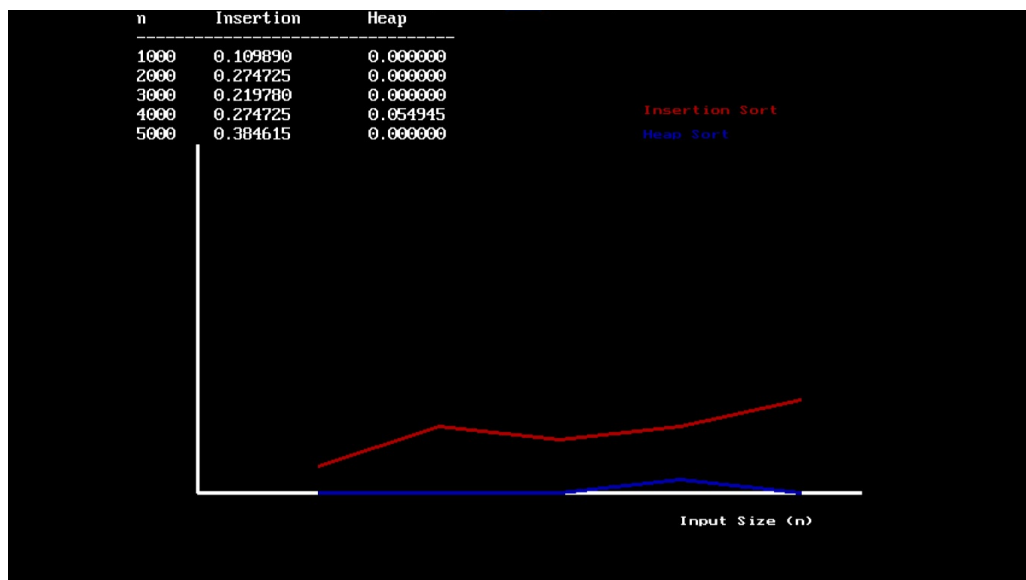
```

}

setcolor(RED);
for(i=0;i<idx-1;i++)
    line(x[i],yIns[i],x[i+1],yIns[i+1]);
setcolor(BLUE);
for(i=0;i<idx-1;i++)
    line(x[i],yHeap[i],x[i+1],yHeap[i+1]);
setcolor(RED);
outtextxy(420,80,"Insertion Sort");
setcolor(BLUE);
outtextxy(420,100,"Heap Sort");
getch();
closegraph();
}

```

OUTPUT



PROGRAM

```
#include <stdio.h>

#include <conio.h>

#define MAX 20

int queue[MAX],front=-1,rear=-1;
int visited[MAX],graph[MAX][MAX];
int n;

void enqueue(int item)
{
    if(rear==MAX-1)
        printf("\nQueue Overflow");
    else
    {
        if(front==-1)
            front=0;
        queue[++rear]=item;
    }
}

int dequeue()
{
    int item;
    if(front==-1 || front>rear)
        return -1;
    item=queue[front++];
    return item;
}

void bfs(int start)
{
    int i,current;
```

```

enqueue(start);
visited[start]=1;
while(front<=rear)
{
    current=dequeue();
    printf("%d ",current);
    for(i=0;i<n;i++)
    {
        if(graph[current][i]==1 && visited[i]==0)
        {
            enqueue(i);
            visited[i]=1;
        }
    }
}
}

void main()
{
    int i,j,start;
    clrscr();
    printf("\nEnter number of vertices: ");
    scanf("%d",&n);
    printf("\nEnter adjacency matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&graph[i][j]);
        }
    }
}

```

```
}  
for(i=0;i<n;i++)  
    visited[i]=0;  
printf("\nEnter starting vertex: ");  
scanf("%d",&start);  
printf("\nBFS Traversal: ");  
bfs(start);  
getch();  
}
```

OUTPUT

```
Enter number of vertices:  
4  
  
Enter adjacency matrix:  
0 1 1 0  
1 0 0 1  
1 0 0 1  
0 1 1 0  
  
Enter starting vertex: 0  
BFS Traversal: 0 1 2 3 _
```


PROGRAM

```
#include <stdio.h>

#include <conio.h>

int a[10][10],visited[10],n;

void dfs(int v)
{
    int i;
    printf("%d ",v);
    visited[v]=1;
    for(i=1;i<=n;i++)
    {
        if(a[v][i]==1 && visited[i]==0)
        {
            dfs(i);
        }
    }
}

void main()
{
    int i,j,start;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    printf("Enter adjacency matrix:\n");
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            scanf("%d",&a[i][j]);
```

```
        }  
    }  
    for(i=1;i<=n;i++)  
        visited[i]=0;  
    printf("Enter starting vertex: ");  
    scanf("%d",&start);  
    printf("DFS Traversal: ");  
    dfs(start);  
    getch();  
}
```

OUTPUT

A screenshot of a terminal window with a black background and white text. The text shows the execution of a DFS program. It starts with the prompt 'Enter number of vertices:' followed by the input '4'. Then it prompts 'Enter adjacency matrix:' followed by four lines of input: '0 1 1 0', '1 0 0 1', '1 0 0 1', and '0 1 1 0'. Next, it prompts 'Enter starting vertex:' followed by the input '1'. Finally, it displays the output 'DFS Traversal: 1 2 4 3'.

```
Enter number of vertices:  
4  
Enter adjacency matrix:  
0 1 1 0  
1 0 0 1  
1 0 0 1  
0 1 1 0  
Enter starting vertex: 1  
DFS Traversal: 1 2 4 3
```

PROGRAM

```
#include <stdio.h>

#include <conio.h>

#define MAX 10

#define INF 9999

void main()

{

    int cost[MAX][MAX],distance[MAX],visited[MAX];

    int n,i,j,start,count,min,next;

    clrscr();

    printf("Enter number of vertices: ");

    scanf("%d",&n);

    printf("Enter cost matrix:\n");

    for(i=1;i<=n;i++)

    {

        for(j=1;j<=n;j++)

        {

            scanf("%d",&cost[i][j]);

            if(cost[i][j]==0)

                cost[i][j]=INF;

        }

    }

    printf("Enter starting vertex: ");

    scanf("%d",&start);

    for(i=1;i<=n;i++)

    {

        distance[i]=cost[start][i];

        visited[i]=0;

    }
```

```
distance[start]=0;
visited[start]=1;
count=1;
while(count<n)
{
    min=INF;
    for(i=1;i<=n;i++)
    {
        if(distance[i]<min && !visited[i])
        {
            min=distance[i];
            next=i;
        }
    }
    visited[next]=1;
    for(i=1;i<=n;i++)
    {
        if(!visited[i])
        {
            if(min+cost[next][i]<distance[i])
            {
                distance[i]=min+cost[next][i];
            }
        }
    }
    count++;
}
printf("\nShortest distances from vertex %d:\n",start);
for(i=1;i<=n;i++)
```

```
{  
    printf("Distance to %d = %d\n",i,distance[i]);  
}  
getch();  
}
```

OUTPUT

```
Enter number of vertices: 4  
Enter cost matrix:  
0 2 4 0  
2 0 1 7  
4 1 0 3  
0 7 3 0  
Enter starting vertex: 1  
  
Shortest distances from vertex 1:  
Distance to 1 = 0  
Distance to 2 = 2  
Distance to 3 = 3  
Distance to 4 = 6
```

PROGRAM

```
#include <stdio.h>

#include <conio.h>

#define MAX 10

#define INF 9999

void main()

{

    int cost[MAX][MAX],visited[MAX];

    int n,i,j,min,a,b,u,v;

    int ne=1;

    int mincost=0;

    clrscr();

    printf("Enter number of vertices: ");

    scanf("%d",&n);

    printf("Enter the cost adjacency matrix:\n");

    for(i=1;i<=n;i++)

    {

        for(j=1;j<=n;j++)

        {

            scanf("%d",&cost[i][j]);

            if(cost[i][j]==0)

                cost[i][j]=INF;

        }

    }

    for(i=1;i<=n;i++)

        visited[i]=0;

    visited[1]=1;

    printf("\nEdges in Minimum Spanning Tree:\n");

    while(ne<n)
```

```

{
    min=INF;
    for(i=1;i<=n;i++)
    {
        if(visited[i])
        {
            for(j=1;j<=n;j++)
            {
                if(!visited[j] && cost[i][j]<min)
                {
                    min=cost[i][j];
                    a=u=i;
                    b=v=j;
                }
            }
        }
    }

    printf("%d edge (%d -> %d) = %d\n",ne++,a,b,min);
    mincost+=min;
    visited[b]=1;
}

printf("\nMinimum cost = %d\n",mincost);
getch();
}

```

OUTPUT

```
Enter number of vertices: 4
Enter the cost adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0

Edges in Minimum Spanning Tree:
1 edge (1 -> 2) = 2
2 edge (2 -> 3) = 3
3 edge (1 -> 4) = 6

Minimum cost = 11
```


PROGRAM

```
#include <stdio.h>

#include <conio.h>

#define MAX 10

#define INF 9999

void floydWarshall(int graph[MAX][MAX],int n)
{
    int dist[MAX][MAX];
    int i,j,k;
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            dist[i][j]=graph[i][j];
        }
    }
    for(k=0;k<n;k++)
    {
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
            {
                if(dist[i][j]>dist[i][k]+dist[k][j])
                {
                    dist[i][j]=dist[i][k]+dist[k][j];
                }
            }
        }
    }
}
```

```

printf("\nShortest distance matrix:\n");
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        if(dist[i][j]==INF)
            printf("INF ");
        else
            printf("%d ",dist[i][j]);
    }
    printf("\n");
}

void main()
{
    int graph[MAX][MAX],n,i,j;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    printf("Enter the adjacency matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&graph[i][j]);
        }
    }
    floydWarshall(graph,n);
}

```

OUTPUT

```
Enter number of vertices: 4
Enter the adjacency matrix:
2 0 1 7
4 0 3 4
1 9 3 6
2 5 2 4
```

```
Shortest distance matrix:
2 0 1 4
4 0 3 4
1 1 2 5
2 2 2 4
```

PROGRAM

```
#include<stdio.h>

#include<conio.h>

#define MAX 10

void main()

{

    int reach[MAX][MAX],n;

    int i,j,k;

    clrscr();

    printf("Enter number of vertices: ");

    scanf("%d",&n);

    printf("Enter adjacency matrix:\n");

    for(i=0;i<n;i++)

    {

        for(j=0;j<n;j++)

        {

            scanf("%d",&reach[i][j]);

        }

    }

    for(k=0;k<n;k++)

    {

        for(i=0;i<n;i++)

        {

            for(j=0;j<n;j++)

            {

                reach[i][j]=reach[i][j]||(reach[i][k]&&reach[k][j]);

            }

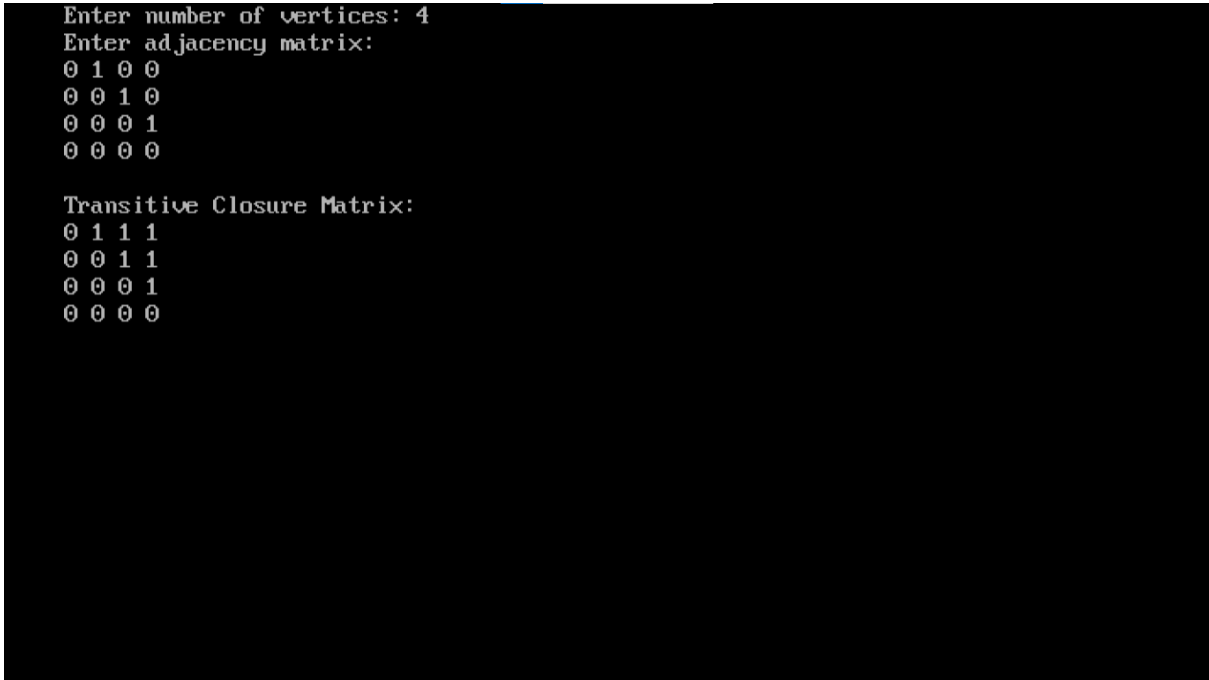
        }

    }

}
```

```
printf("\nTransitive Closure Matrix:\n");  
for(i=0;i<n;i++)  
{  
    for(j=0;j<n;j++)  
    {  
        printf("%d ",reach[i][j]);  
    }  
    printf("\n");  
}  
getch();  
}
```

OUTPUT



```
Enter number of vertices: 4  
Enter adjacency matrix:  
0 1 0 0  
0 0 1 0  
0 0 0 1  
0 0 0 0  
  
Transitive Closure Matrix:  
0 1 1 1  
0 0 1 1  
0 0 0 1  
0 0 0 0
```

PROGRAM

```
#include<stdio.h>

#include<conio.h>

int min, max;

void minmax(int a[], int low, int high)
{
    int mid, min1, max1;
    if(low == high)
    {
        min = max = a[low];
    }
    else if(low == high-1)
    {
        if(a[low] < a[high])
        {
            min = a[low];
            max = a[high];
        }
        else
        {
            min = a[high];
            max = a[low];
        }
    }
    else
    {
        mid = (low + high)/2;
        minmax(a, low, mid);
        min1 = min;
```

```
        max1 = max;
        minmax(a, mid+1, high);
        if(min > min1)
            min = min1;
        if(max < max1)
            max = max1;
    }
}

void main()
{
    int a[20], n, i;
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter the elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    minmax(a,0,n-1);
    printf("\nMinimum element = %d",min);
    printf("\nMaximum element = %d",max);
    getch();
}
```

OUTPUT

```
Enter number of elements: 5
Enter the elements:
12
45
7
23
56

Minimum element = 7
Maximum element = 56
```


PROGRAM

```
#include<stdio.h>

#include<conio.h>

#include<dos.h>

#include<graphics.h>

#include<time.h>

#include<stdlib.h>

#define MAX 6000

int a[MAX],b[MAX];

void merge(int a[],int l,int m,int r)
{
    int i=l,j=m+1,k=l,temp[MAX];
    while(i<=m && j<=r)
    {
        if(a[i]<a[j])
            temp[k++]=a[i++];
        else
            temp[k++]=a[j++];
    }
    while(i<=m)
        temp[k++]=a[i++];
    while(j<=r)
        temp[k++]=a[j++];
    for(i=l;i<=r;i++)
        a[i]=temp[i];
}

void mergesort(int a[],int l,int r)
{
    int m;
```

```

    if(l<r)
    {
        m=(l+r)/2;
        mergesort(a,l,m);
        mergesort(a,m+1,r);
        merge(a,l,m,r);
    }
}

int partition(int a[],int low,int high)
{
    int pivot=a[high];
    int i=low-1,j,temp;
    for(j=low;j<high;j++)
    {
        if(a[j]<=pivot)
        {
            i++;
            temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
    }
    temp=a[i+1];
    a[i+1]=a[high];
    a[high]=temp;
    return i+1;
}

void quicksort(int a[],int low,int high)
{

```

```

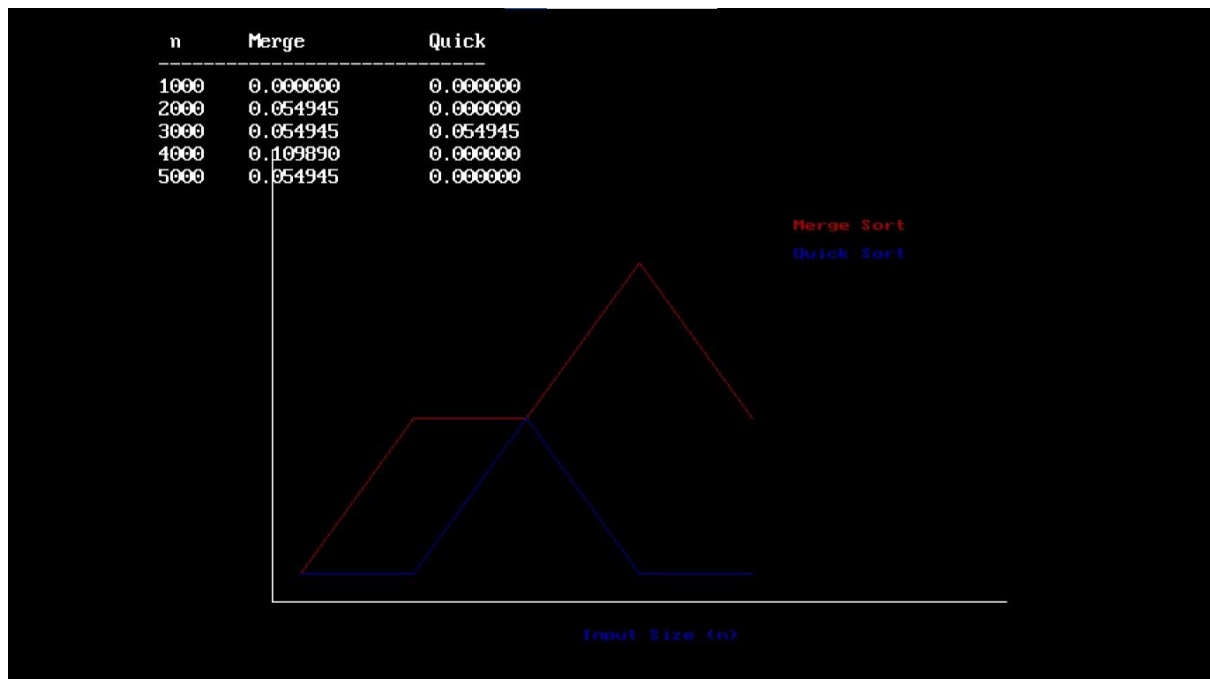
int pi;
if(low<high)
{
    pi=partition(a,low,high);
    quicksort(a,low,pi-1);
    quicksort(a,pi+1,high);
}
}

void main()
{
    int n,i;
    int gd=DETECT,gm;
    float merge_time,quick_time;
    clock_t start,end;
    int x[5],y1[5],y2[5];
    int index=0;
    initgraph(&gd,&gm,"C:\\TURBOC3\\BGI");
    printf("\n n\tMerge\t\tQuick\n");
    printf("-----\n");
    for(n=1000;n<=5000;n+=1000)
    {
        for(i=0;i<n;i++)
        {
            a[i]=rand()%1000;
            b[i]=a[i];
        }
        start=clock();
        mergesort(a,0,n-1);
        end=clock();
    }
}

```

```
merge_time=(float)(end-start)/CLK_TCK;
start=clock();
quicksort(b,0,n-1);
end=clock();
quick_time=(float)(end-start)/CLK_TCK;
printf("%d\t%f\t%f\n",n,merge_time,quick_time);
x[index]=100+index*80;
y1[index]=400-(merge_time*2000);
y2[index]=400-(quick_time*2000);
index++;
}
line(80,420,600,420);
line(80,420,80,100);
for(i=0;i<4;i++)
{
    setcolor(RED);
    line(x[i],y1[i],x[i+1],y1[i+1]);
    setcolor(BLUE);
    line(x[i],y2[i],x[i+1],y2[i+1]);
}
setcolor(RED);
outtextxy(450,150,"Merge Sort");
setcolor(BLUE);
outtextxy(450,170,"Quick Sort");
outtextxy(300,440,"Input Size (n)");
getch();
}
```

OUTPUT



PROGRAM

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

int x[20],count=0;

int place(int k,int i)

{

    int j;

    for(j=1;j<k;j++)

    {

        if((x[j]==i) || (abs(x[j]-i)==abs(j-k)))

            return 0;

    }

    return 1;

}

void nqueen(int k,int n)

{

    int i,j;

    for(i=1;i<=n;i++)

    {

        if(place(k,i))

        {

            x[k]=i;

            if(k==n)

            {

                count++;

                printf("\nSolution %d:\n\n",count);

                for(i=1;i<=n;i++)

                {
```

```

        for(j=1;j<=n;j++)
        {
            if(x[i]==j)
                printf(" Q ");
            else
                printf(" - ");
        }
        printf("\n");
    }
}

else
{
    nqueen(k+1,n);
}
}
}

void main()
{
    int n;

    clrscr();

    printf("Enter number of Queens: ");

    scanf("%d",&n);

    nqueen(1,n);

    if(count==0)

        printf("No solution exists");

    getch();
}

```

OUTPUT

```
Enter number of Queens: 4
```

```
Solution 1:
```

```
-  Q  -  -  
-  -  -  Q  
Q  -  -  -  
-  -  Q  -
```

```
Solution 2:
```

```
-  -  Q  -  
Q  -  -  -  
-  -  -  Q  
-  Q  -  -
```


PROGRAM

```
#include<stdio.h>

#include<conio.h>

#define MAX 10

#define INF 999

int n;

int cost[MAX][MAX];

int visited[MAX];

int path[MAX];

int mincost = INF;

int min(int a,int b)
{
    if(a<b) return a;

    return b;
}

void tsp(int city,int count,int cost_so_far)
{
    int i;

    if(count==n && cost[city][0]>0)
    {
        mincost = min(mincost,cost_so_far + cost[city][0]);

        return;
    }

    for(i=0;i<n;i++)
    {
        if(!visited[i] && cost[city][i]>0)
        {
            visited[i]=1;

            tsp(i,count+1,cost_so_far + cost[city][i]);
        }
    }
}
```

```

        visited[i]=0;
    }
}
}
int nearestNeighbor()
{
    int i,current=0;
    int total=0;
    int visited2[MAX]={0};
    visited2[current]=1;
    for(i=0;i<n-1;i++)
    {
        int j,next=-1,min=INF;
        for(j=0;j<n;j++)
        {
            if(!visited2[j] && cost[current][j]<min)
            {
                min=cost[current][j];
                next=j;
            }
        }
        visited2[next]=1;
        total += min;
        current=next;
    }
    total += cost[current][0];
    return total;
}
void main()

```

```
{
    int i,j;
    int approx, error;
    clrscr();
    printf("Enter number of cities: ");
    scanf("%d",&n);
    printf("Enter cost matrix:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&cost[i][j]);
        }
    }
    for(i=0;i<n;i++)
        visited[i]=0;
    visited[0]=1;
    tsp(0,1,0);
    approx = nearestNeighbor();
    error = approx - mincost;
    printf("\nOptimal Cost = %d",mincost);
    printf("\nApproximate Cost = %d",approx);
    printf("\nError in Approximation = %d",error);
    getch();
}
```

OUTPUT

```
Enter number of cities: 4
Enter cost matrix:
0 10 20 35
25 15 0 35
10 15 20 25
35 40 10 5

Optimal Cost = 65
Approximate Cost = 70
Error in Approximation = 5
```

PROGRAM

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#include<time.h>

void swap(int *a,int *b)

{

    int temp;

    temp=*a;

    *a=*b;

    *b=temp;

}

int partition(int a[],int l,int r)

{

    int pivot=a[r];

    int i=l-1,j;

    for(j=l;j<r;j++)

    {

        if(a[j]<=pivot)

        {

            i++;

            swap(&a[i],&a[j]);

        }

    }

    swap(&a[i+1],&a[r]);

    return i+1;

}

int randomPartition(int a[],int l,int r)

{
```

```

    int i = 1 + rand()%(r-l+1);
    swap(&a[i],&a[r]);
    return partition(a,l,r);
}

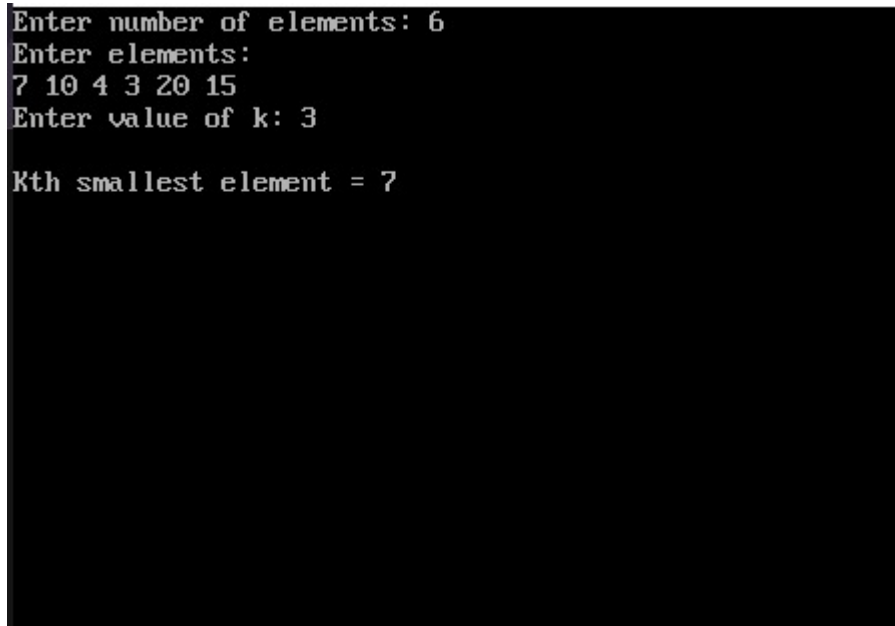
int kthSmallest(int a[],int l,int r,int k)
{
    if(k>0 && k<=r-l+1)
    {
        int pos=randomPartition(a,l,r);
        if(pos-l == k-1)
            return a[pos];
        if(pos-l > k-1)
            return kthSmallest(a,l,pos-l,k);
        return kthSmallest(a,pos+1,r,k-pos+1-1);
    }
    return -1;
}

void main()
{
    int n,k,i;
    int a[20];
    clrscr();
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("Enter value of k: ");
    scanf("%d",&k);

```

```
    srand(time(NULL));  
    printf("\nKth smallest element = %d",kthSmallest(a,0,n-1,k));  
    getch();  
}
```

OUTPUT

A screenshot of a terminal window with a black background and white text. The text shows the user input for a program: 'Enter number of elements: 6', 'Enter elements: 7 10 4 3 20 15', 'Enter value of k: 3', and the program output 'Kth smallest element = 7'.

```
Enter number of elements: 6  
Enter elements:  
7 10 4 3 20 15  
Enter value of k: 3  
  
Kth smallest element = 7
```